



HABIB UNIVERSITY

Data Structures & Algorithms

CS/CE 102/171 Spring 2025

Instructor: Maria Samad

Quiz # 2 Solution

QUESTION 1: ALGORITHM DESIGN USING BINARY SEARCH (10 marks)

- **Finding the middle location of the given target value in an array of numbers using BINARY SEARCH:**
 - A fixed sized array is given that contains n integers, where 'n' > 1, while the rest of the elements in array are None, if needed. Assume both n and array are in the correct format, so you may use them directly WITHOUT taking any inputs. However, they must be declared/defined in the algorithm properly.
 - The user inputs a target value that is a number, which needs to be searched in the array
 - Design an algorithm that uses binary search to find the middle index of the target value.
 - For example:
 - If given array is [2, 3, 3, 3, 7, 7, 9, None], and given target is 3, then the middle occurrence index of 3 will be the one on index 2 in the array, so the algorithm should return 2
 - If target is not found, then it should just return -1
 - Similarly, if the given target is 7 for the above-mentioned array, then middle occurrence index of 7 is the floor value of middle, and it will return 4
- **REQUIREMENTS:**
 - Do not **HARD CODE** your design for the given example. The algorithm should work for any valid sized arrays.
 - Declare all the data structures and variables being used in this algorithm, including any inputs, if any.
 - **DO NOT JUST SAY “PERFORM BINARY SEARCH TO DO ...”, ETC. INSTEAD SPECIFY THE ACTUAL LOGIC FOR IT ON HOW IT IS BEING IMPLEMENTED HERE!**
 - You **MUST NOT** use any additional data structure (including lists/arrays) apart from the given input array.
 - You **MUST NOT** modify the array to forcefully convert it to a Python list just to perform your binary search algorithm
 - You **MUST USE BINARY SEARCH, OTHERWISE, NO MARKS WILL BE AWARDED IF ANY OTHER SEARCH IS USED.**

SOLUTION:

- `n` – an integer variable to represent the valid number of elements in the given array
- `arr1` – the input array containing `n` integers
- `target` – take integer input from the user, which is the value to be searched in the given input array

Apply binary search to find the first occurrence of target value in the given array

- `left = 0`
- `right = n - 1`
- `first_index = None`

Updating the indices to fetch the first occurrence of target value

- while `left <= right`, do the following:
 - `mid = (left + right) // 2`
 - if `arr1[mid]` is equal to the target value, then do as follows:
 - `first_index = mid`
 - `right = mid - 1`
 - else if `arr1[mid]` is less than the target value, then do the following:
 - `left = mid + 1`
 - otherwise, do as follows:
 - `right = mid - 1`
- if `first_index` is still `None`, then it means index must not have been found in the above loop, so do the following:
 - `return -1`

Otherwise the first index has been found, so apply binary search again to find the last occurrence of the target value

- `left = 0`
- `right = n - 1`
- `last_index = first_index`

Updating the indices to fetch the last occurrence of target value

- while `left <= right`, do the following:
 - `mid = (left + right) // 2`
 - if `arr1[mid]` is equal to the target value, then do as follows:
 - `last_index = mid`
 - `left = mid + 1`
 - else if `arr1[mid]` is less than the target value, then do the following:
 - `left = mid + 1`
 - otherwise, do as follows:
 - `right = mid - 1`

Now calculate the middle occurrence index and return that index

- `middle_index = (first_index + last_index) // 2`
- `return middle_index`

QUESTION 2: PRIORITY QUEUE OPERATIONS (10 marks)

A priority queue can be implemented using fixed-sized arrays or lists. Assume the given priority queue is implemented using fixed-sized arrays. The largest number represents the highest priority for the given Priority Queues. For a given Priority Queue, called PQ with size = 4 and empty slots represented as -1, complete the following table:

OPERATION	RETURN VALUE	UNSORTED PQ CONTENTS	TIME COMPLEXITY	SORTED PQ CONTENTS	TIME COMPLEXITY
PQ.is_full()	False	[-1, -1, -1, -1]	O(1)	[-1, -1, -1, -1]	O(1)
PQ.enqueue(37, "D")	–	[(37, D), -1, -1, -1]	O(1)	[(37, D), -1, -1, -1]	O(1)
PQ.enqueue(17, "S")	–	[(37, D), (17, S), -1, -1]	O(1)	[(37, D), (17, S), -1, -1]	O(n)
PQ.is_empty()	False	[(37, D), (17, S), -1, -1]	O(1)	[(37, D), (17, S), -1, -1]	O(1)
PQ.enqueue(57, "A")	–	[(37, D), (17, S), (57, A), -1]	O(1)	[(57, A), (37, D), (17, S), -1]	O(n)
PQ.dequeue()	(57, A)	[(37, D), (17, S), -1, -1]	O(n)	[(37, D), (17, S), -1, -1]	O(n)
PQ.dequeue()	(37, D)	[(17, S), -1, -1, -1]	O(n)	[(17, S), -1, -1, -1]	O(n)
PQ.enqueue(27, "I")	–	[(17, S), (27, I), -1, -1]	O(1)	[(27, I), (17, S), -1, -1]	O(n)
PQ.peek()	(27, I)	[(17, S), (27, I), -1, -1]	O(n)	[(27, I), (17, S), -1, -1]	O(1)
PQ.enqueue(20, "S")	–	[(17, S), (27, I), (20, S), -1]	O(1)	[(27, I), (20, S), (17, S), -1]	O(n)
PQ.enqueue(35, "F")	–	[(17, S), (27, I), (20, S), (35, F)]	O(1)	[(35, F), (27, I), (20, S), (17, S)]	O(n)
PQ.dequeue()	(35, F)	[(17, S), (27, I), (20, S), -1]	O(n)	[(27, I), (20, S), (17, S), -1]	O(n)
PQ.peek()	(27, I)	[(17, S), (27, I), (20, S), -1]	O(n)	[(27, I), (20, S), (17, S), -1]	O(1)
PQ.enqueue(2, "U")	–	[(17, S), (27, I), (20, S), (2, U)]	O(1)	[(27, I), (20, S), (17, S), (2, U)]	O(n)
PQ.enqueue(41, "N")	Error	[(17, S), (27, I), (20, S), (2, U)]	O(1)	[(27, I), (20, S), (17, S), (2, U)]	O(1)

QUESTION 3: SELECTION SORT (10 marks)

Sort the given list in descending order using selection sort algorithm.

- [33, 12, 41, 94, 27, 65, 79, 16, 58, 90, 2]

SHOW ALL INTERIM CALCULATIONS IN ORDER TO GET COMPLETE/FULL MARKS. An answer without the intermediate steps will directly result in 0 marks.

Q3
Date: Selection Sort

[33, 12, 41, 94, 27, 65, 79, 16, 58, 90, 2]

max = 94 [94, 12, 41, 33, 27, 65, 79, 16, 58, 90, 2]
sorted

max = 90 [94, 90, 41, 33, 27, 65, 79, 16, 58, 12, 2]
sorted

max = 79 [94, 90, 79, 33, 27, 65, 41, 16, 58, 12, 2]
sorted

max = 65 [94, 90, 79, 65, 27, 33, 41, 16, 58, 12, 2]
sorted

max = 58 [94, 90, 79, 65, 58, 33, 41, 16, 27, 12, 2]
sorted

max = 41 [94, 90, 79, 65, 58, 41, 33, 16, 27, 12, 2]
sorted

max = 33 [94, 90, 79, 65, 58, 41, 33, 16, 27, 12, 2]
sorted

max = 27 [94, 90, 79, 65, 58, 41, 33, 27, 16, 12, 2]
sorted

max = 16 [94, 90, 79, 65, 58, 41, 33, 27, 16, 12, 2]
sorted

max = 12 [94, 90, 79, 65, 58, 41, 33, 27, 16, 12, 2]
sorted

max = 2 [94, 90, 79, 65, 58, 41, 33, 27, 16, 12, 2]
sorted

SORTED!

DALMATIAN